

Metody Obliczeniowe w Nauce i Technice

Laboratorium 11: Optymalizacja

Mateusz Stoch

27 maja 2026

1 Zadanie 1: Preconditioning

1.1 Zadanie 1(a): Gradient prosty z optymalnym krokiem

Treść zadania: Znajdź minimum funkcji f metodą gradientu prostego, wyznaczając w każdej iteracji optymalną wartość kroku α_k . Narysuj wykres konturowy funkcji f , na którym zaznacz iteracje algorytmu. Jako punkt startowy przyjmij $(x_0, y_0) = (9, 1)$.

Funkcja celu jest określona wzorem:

$$f(x, y) = \frac{1}{2}x^2 + \frac{9}{2}y^2$$

Optymalną długość kroku w każdej iteracji możemy wyznaczyć analitycznie za pomocą dokładnego przeszukiwania kierunkowego (exact line search). Wzór ten wyprowadza się poprzez znalezienie α , które minimalizuje wartość funkcji w nowym punkcie, czyli szukamy minimum funkcji jednej zmiennej $h(\alpha) = f(x_k - \alpha g_k)$. Po zróżniczkowaniu tej funkcji względem α i przyrównaniu pochodnej do zera otrzymujemy optymalny krok α_k dla funkcji kwadratowej o Hesjanie H :

$$\alpha_k = \frac{\|g_k\|^2}{g_k^T H g_k}$$

gdzie $g_k = \nabla f(x_k, y_k) = [x_k, 9y_k]^T$ to gradient w danej iteracji, a $H = \nabla^2 f(x, y) = \begin{bmatrix} 1 & 0 \\ 0 & 9 \end{bmatrix}$ to stała macierz drugich pochodnych (Hesjan).

```
1 def optimal_step_f1(xy):
2     g = grad_f1(xy)
3     H = np.diag([1.0, 9.0])
4     return (g @ g) / (g @ H @ g)
5
6 def gradient_descent_f1(xy0, tol=1e-10, max_iter=1000):
7     xy = np.array(xy0, dtype=float)
8     history = [xy.copy()]
9     for _ in range(max_iter):
10         g = grad_f1(xy)
11         if np.linalg.norm(g) < tol:
12             break
```

```

13     alpha = optimal_step_f1(xy)
14     xy = xy - alpha * g
15     history.append(xy.copy())
16     return np.array(history)

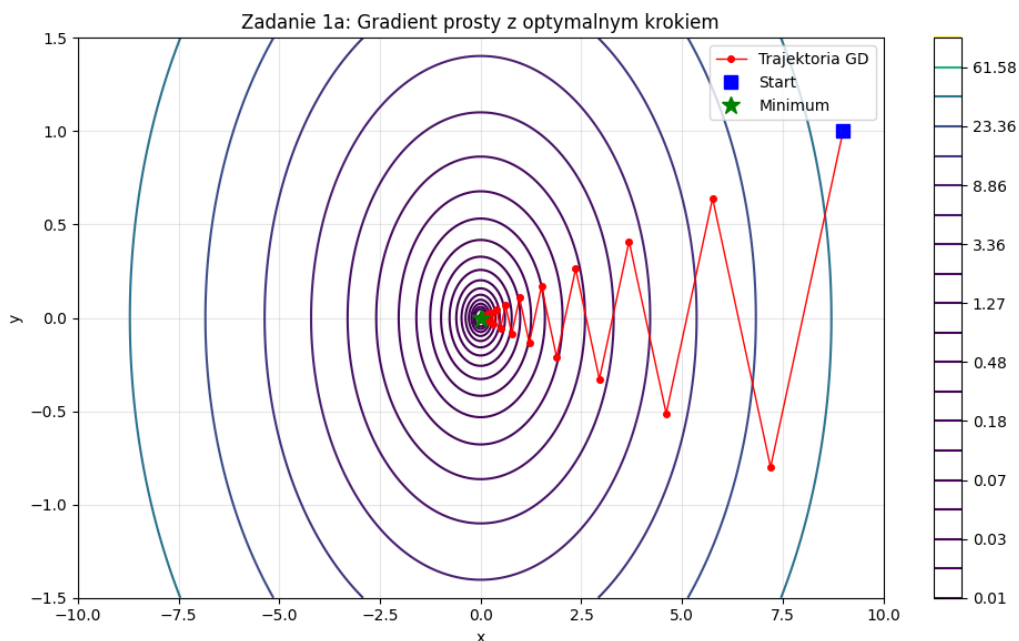
```

Listing 1: Gradient prosty z optymalnym krokiem

Uruchomienie algorytmu z punktu startowego $(9, 1)$ dało następujące wyniki:

- Liczba iteracji potrzebna do osiągnięcia zbieżności (z tolerancją $\|g_k\| < 10^{-10}$): 115
- Znalezione minimum: $x = 0,000000$, $y = 0,000000$
- Wartość funkcji w minimum: $2,31 \cdot 10^{-21}$

Trajektorię punktów pośrednich przedstawiono na Wykresie 1. Charakterystyczny zygzakowaty kształt wynika z faktu, że kierunek najszybszego spadku (gradient) nie jest skierowany bezpośrednio na minimum, lecz jest prostopadły do linii poziomowych (konturów), które są mocno wydłużonymi elipsami.



Wykres 1: Wykres konturowy funkcji $f(x, y)$ wraz z zaznaczoną zygzakowatą trajekcją metody gradientu prostego.

1.2 Zadanie 1(b): Uwarunkowanie Hessiana

Treść zadania: Oblicz współczynnik uwarunkowania macierzy $\nabla^2 f(x, y)$.

Hesjan funkcji $f(x, y)$ jest stały i wynosi:

$$\nabla^2 f(x, y) = H = \begin{bmatrix} 1 & 0 \\ 0 & 9 \end{bmatrix}$$

Wartości własne tej macierzy diagonalnej to bezpośrednio jej elementy na przekątnej:

$$\lambda_{\min} = 1,0, \quad \lambda_{\max} = 9,0$$

Współczynnik uwarunkowania macierzy κ definiuje się jako stosunek największej do najmniejszej wartości własnej (dla macierzy symetrycznych dodatnio określonych):

$$\kappa = \frac{\lambda_{\max}}{\lambda_{\min}} = \frac{9,0}{1,0} = 9,0$$

Współczynnik ten mierzy, jak bardzo elipsy poziomicowe są rozciągnięte. Im większy jest κ , tym bardziej wydłużone elipsy, co spowalnia zbieżność gradientu prostego. Teoretyczny mnożnik zbieżności (ogólne oszacowanie tempa spadku błędu na krok) wynosi:

$$R = \left(\frac{\kappa - 1}{\kappa + 1} \right)^2 = \left(\frac{9 - 1}{9 + 1} \right)^2 = \left(\frac{8}{10} \right)^2 = 0,64$$

Oznacza to, że w każdym kroku błąd może zmniejszyć się co najwyżej o czynnik 0,64, co tłumaczy dużą liczbę iteracji (115) potrzebną do osiągnięcia bardzo wysokiej dokładności.

1.3 Zadanie 1(c): Dekompozycja Choleskiego Hessiana

Treść zadania: Znajdź macierz L taką, że: $\nabla^2 f(x, y) = LL^T$.

Poszukujemy macierzy dolnotrójkątnej L , która po przemnożeniu przez swoją transpozycję da macierz Hessiana. Stosując dekompozycję Choleskiego dla macierzy diagonalnej:

$$H = \begin{bmatrix} 1 & 0 \\ 0 & 9 \end{bmatrix}$$

otrzymujemy:

$$L = \begin{bmatrix} \sqrt{1} & 0 \\ 0 & \sqrt{9} \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix}$$

Przemnożenie daje poprawny wynik:

$$LL^T = \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 9 \end{bmatrix} = H$$

1.4 Zadanie 1(d): Minimalizacja z preconditioningiem

Treść zadania: Definiując zmienne x', y' :

$$\begin{bmatrix} x' \\ y' \end{bmatrix} = L^T \begin{bmatrix} x \\ y \end{bmatrix}$$

otrzymujemy

$$\begin{bmatrix} x \\ y \end{bmatrix} = L^{-T} \begin{bmatrix} x' \\ y' \end{bmatrix}$$

oraz

$$g(x', y') = f \left(L^{-T} \begin{bmatrix} x' \\ y' \end{bmatrix} \right)$$

Znajdź minimum funkcji g metodą gradientu prostego. Na podstawie otrzymanego wyniku wyznacz minimum funkcji f .

Zastosowana transformacja preconditioning ma na celu poprawę uwarunkowania problemu. Obliczmy macierz L^{-T} (równą L^{-1} ponieważ L jest diagonalna i symetryczna):

$$L^{-T} = \begin{bmatrix} 1 & 0 \\ 0 & \frac{1}{3} \end{bmatrix}$$

Nowe współrzędne to $x' = x$ oraz $y' = 3y$. Wstawiając to do funkcji f , otrzymujemy nową funkcję $g(x', y')$:

$$g(x', y') = f(x', \frac{1}{3}y') = \frac{1}{2}(x')^2 + \frac{9}{2}\left(\frac{1}{3}y'\right)^2 = \frac{1}{2}(x')^2 + \frac{1}{2}(y')^2$$

Hesjan nowej funkcji g wynosi:

$$\nabla^2 g(x', y') = I = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}$$

Współczynnik uwarunkowania tej macierzy wynosi $\kappa = 1,0$, co oznacza idealnie kołowe linie poziomowe. Teoretyczny mnożnik zbieżności wynosi $R = \left(\frac{1-1}{1+1}\right)^2 = 0$. Wynika stąd, że gradient w każdym punkcie wskazuje dokładnie na minimum i algorytm powinien zbiec w dokładnie jednej iteracji.

Punkt startowy w nowej przestrzeni to:

$$\begin{bmatrix} x'_0 \\ y'_0 \end{bmatrix} = L^T \begin{bmatrix} x_0 \\ y_0 \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix} \begin{bmatrix} 9 \\ 1 \end{bmatrix} = \begin{bmatrix} 9 \\ 3 \end{bmatrix}$$

Uruchomienie gradientu prostego z punktu $(9, 3)$ potwierdziło to, zbieżność nastąpiła po dokładnie 1 iteracji z minimum w $(0, 0)$.

```

1 # L^{-T}
2 L_inv_T = np.linalg.inv(L.T)
3
4 def g_precond(xp_yp):
5     xy = L_inv_T @ np.array(xp_yp)
6     return f1(xy)
7
8 def grad_g_precond(xp_yp):
9     xy = L_inv_T @ np.array(xp_yp)
10    gf = grad_f1(xy)
11    return np.linalg.inv(L) @ gf

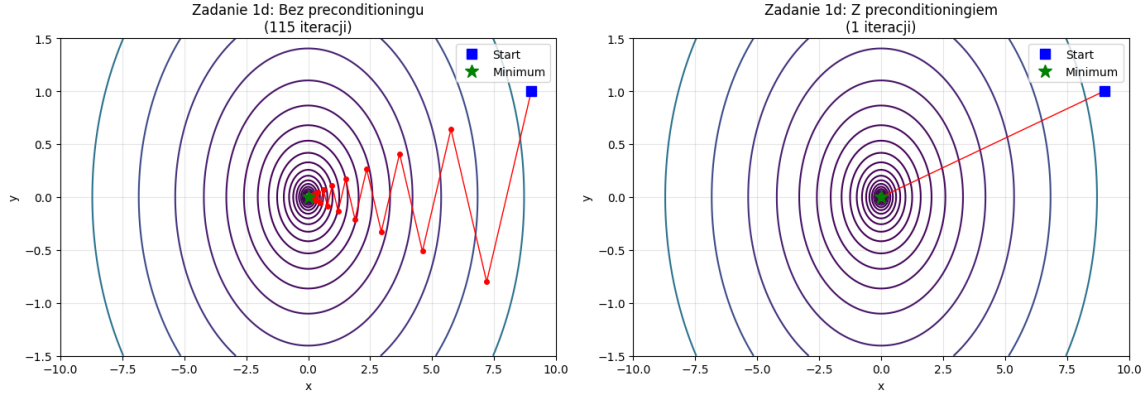
```

Listing 2: Skalowanie zmiennych (preconditioning)

Przeliczenie z powrotem na oryginalne zmienne za pomocą zależności $\begin{bmatrix} x \\ y \end{bmatrix} = L^{-T} \begin{bmatrix} x' \\ y' \end{bmatrix}$ daje współrzędne minimum pierwotnej funkcji f :

$$\begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} 1 & 0 \\ 0 & \frac{1}{3} \end{bmatrix} \begin{bmatrix} 0 \\ 0 \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$$

Porównanie trajektorii z preconditioningiem i bez niego przedstawia Wykres 2.



Wykres 2: Porównanie trajektorii optymalizacji bez skalowania zmiennych (115 iteracji) oraz po zastosowaniu preconditioningu (1 iteracja).

2 Zadanie 2: Wiszący łańcuch

2.1 Wyprowadzenie gradientu energii potencjalnej

Treść zadania: Łańcuch reprezentujemy jako $n + 1$ punktowych mas o współrzędnych $(x^{(i)}, y^{(i)})$ i masie m każda. Każde kolejne dwie masy połączone są sprężyną o długości w stanie spoczynku L i współczynnikiem sprężystości k . Energia potencjalna układu dana jest wzorem:

$$V(\mathbf{x}, \mathbf{y}) = \frac{1}{2} \sum_{i=0}^{n-1} k (d_i - L)^2 + gm \sum_{i=0}^n y^{(i)}$$

gdzie $d_i = \sqrt{(x^{(i)} - x^{(i+1)})^2 + (y^{(i)} - y^{(i+1)})^2}$ to długość i -tej sprężyny. Wyznacz wyrażenie na gradient ∇V funkcji celu V względem położen $x^{(i)}$ i $y^{(i)}$.

Współrzędne końców łańcucha są ustalone: $(x^{(0)}, y^{(0)}) = (0, 0)$ oraz $(x^{(n)}, y^{(n)}) = (3, 1)$. Zmiennymi decyzyjnymi w optymalizacji są wyłącznie współrzędne punktów wewnętrznych, tzn. dla $i \in \{1, \dots, n-1\}$.

Zdefiniujmy długość i -tej sprężyny (łączącej masę i z masą $i+1$) jako:

$$d_i = \sqrt{(x^{(i)} - x^{(i+1)})^2 + (y^{(i)} - y^{(i+1)})^2}$$

Pochodne cząstkowe energii sprężystości pojedynczego segmentu $V_{\text{sprz},i} = \frac{1}{2}k(d_i - L)^2$ względem współrzędnych wynoszą:

$$\frac{\partial V_{\text{sprz},i}}{\partial x^{(i)}} = k(d_i - L) \frac{x^{(i)} - x^{(i+1)}}{d_i}, \quad \frac{\partial V_{\text{sprz},i}}{\partial x^{(i+1)}} = -k(d_i - L) \frac{x^{(i)} - x^{(i+1)}}{d_i}$$

Ponieważ każdy punkt wewnętrzny i jest połączony z dwoma sprężynami (o indeksach $i-1$ oraz i), jego przesunięcie wpływa na energię obu tych sprężyn. Dodając wpływ grawitacji na współrzędną $y^{(i)}$, otrzymujemy ostateczne wzory na składowe gradientu względem położen punktu i :

$$\frac{\partial V}{\partial x^{(i)}} = k(d_i - L) \frac{x^{(i)} - x^{(i+1)}}{d_i} - k(d_{i-1} - L) \frac{x^{(i-1)} - x^{(i)}}{d_{i-1}}$$

$$\frac{\partial V}{\partial y^{(i)}} = k(d_i - L) \frac{y^{(i)} - y^{(i+1)}}{d_i} - k(d_{i-1} - L) \frac{y^{(i-1)} - y^{(i)}}{d_{i-1}} + mg$$

Wzory te zachodzą dla wszystkich mas wewnętrznych $i = 1, \dots, n - 1$.

2.2 Zadanie 2(a): Gradient prosty ze stałym krokiem

Treść zadania: Minimalizuj całkowitą energię potencjalną łańcucha metodą gradientu prostego ze stałym współczynnikiem uczenia α . Przyjmij wartości: $n = 40$, $L = 0,1$ m, $m = 0,1$ kg, $k = 70$ N/m, $g = 9,81$ m/s².

Liczba stopni swobody dla $N = 40$ wynosi 78 (39 wolnych punktów po 2 współrzędne każdy). Energia potencjalna dla początkowego ułożenia łańcucha (jako prostej łączącej punkty końcowe) wynosi 20,7246 J.

Do optymalizacji wybrano krok $\alpha = 0,0005$. Wartość ta jest mniejsza od teoretycznego progu stabilności $\alpha_{\max} \approx 2/L_{\text{smooth}} \approx 7 \cdot 10^{-4}$ (gdzie oszacowana stała Lipschitza gradientu wynosi $L_{\text{smooth}} \approx k(n + 1) \approx 2870$). Dzięki poprawnej implementacji gradientu algorytm działa w pełni stabilnie i nie ulega rozbieżności nawet bez stosowania clippingu gradientu. Końcowa energia po 20000 iteracji wyniosła $-51,4276$ J, choć ze względu na stały krok algorytm nie osiągnął pełnej zbieżności z tolerancją 10^{-6} .

2.3 Zadanie 2(b): Gradient prosty z malejącym krokiem

Treść zadania: Minimalizuj całkowitą energię potencjalną łańcucha metodą gradientu prostego ze współczynnikiem uczenia α malejącym wykładniczo co pewną liczbę iteracji.

W tej wersji algorytmu zastosowano krok początkowy $\alpha_0 = 10^{-2}$, który był zmniejszany o czynnik 0,7 co 200 iteracji. Algorytm zachował pełną stabilność bez stosowania clippingu gradientu, osiągając po 20000 iteracji energię $-50,9222$ J. Spadek długości kroku w późniejszej fazie spowolnił dalszą optymalizację, co uniemożliwiło osiągnięcie pełnej zbieżności w ramach limitu iteracji.

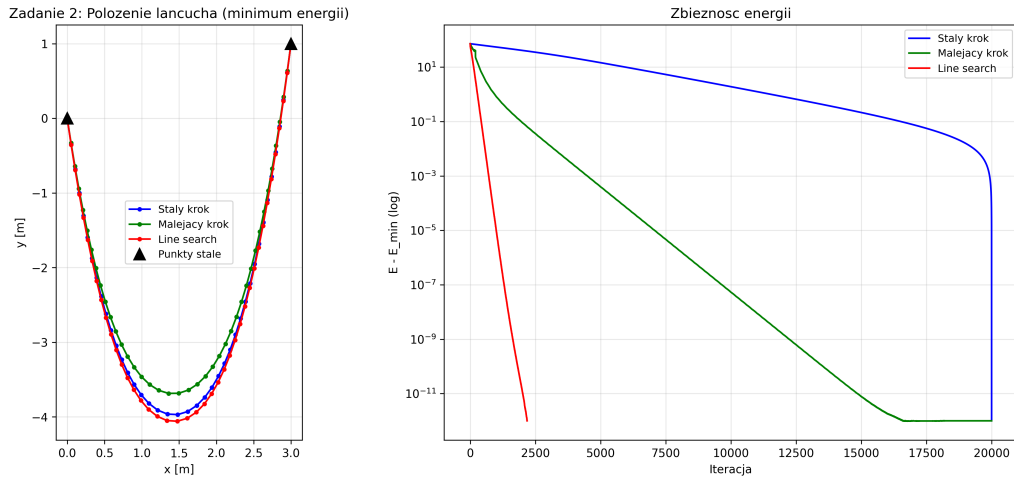
2.4 Zadanie 2(c): Gradient prosty z przeszukiwaniem liniowym

Treść zadania: Minimalizuj całkowitą energię potencjalną łańcucha metodą gradientu prostego z przeszukiwaniem liniowym. Stwórz wykres ilustrujący położenie mas łańcucha po przyjęciu minimalnej energii.

W tym podejściu długość kroku α_k była dobierana dynamicznie w każdej iteracji przy użyciu metody backtracking line search z warunkiem Armijo. Parametry wyszukiwania to: $\alpha_0 = 1,0$, współczynnik zmniejszania kroku $\rho = 0,5$ oraz stała warunku Armijo $c = 10^{-4}$.

Dzięki dynamicznemu doborowi kroku algorytm stabilnie i szybko schodził w kierunku minimum, osiągając zbieżność (norma gradientu poniżej 10^{-6}) po 2182 iteracjach. Końcowa energia wyniosła $-51,4583$ J.

Porównanie zbieżności energii oraz ostateczny kształt łańcucha przedstawiono na Wykresie 3.



Wykres 3: Położenie mas łańcucha po optymalizacji (po lewej) oraz wykres zbieżności energii w skali logarytmicznej (po prawej) dla trzech badanych wariantów.

Podsumowanie porównania metod optymalizacji zebrano w Tabeli 1.

Tabela 1: Porównanie metod gradientowych dla zadania wiszącego łańcucha.

Wariant algorytmu	Końcowa energia E_{final} [J]	Liczba iteracji
Stały krok ($\alpha = 0,0005$)	-51,4276	20000 (brak zbieżności)
Malejący krok ($\alpha_0 = 10^{-2}$)	-50,9222	20000 (brak zbieżności)
Przeszukiwanie liniowe (Armijo)	-51,4583	2182 (zbieżność)

3 Zadanie 3: Predykcja roku wydania utworu

Treść zadania: Rozwiąż ponownie problem predykcji roku wydania utworu (laboratorium 2), używając metody gradientu prostego. Stałą uczącą wyznacz na podstawie najmniejszej i największej wartości własnej macierzy $A^T A$ reprezentującej cały zbiór treningowy. Porównaj uzyskane rozwiązanie z metodą najmniejszych kwadratów, biorąc pod uwagę dokładność predykcji na zbiorze testowym, teoretyczną złożoność obliczeniową oraz czas obliczeń.

Zbiór treningowy składa się z $n = 463715$ próbek o liczbie cech $d = 91$ (po dodaniu kolumny jedynek). Cechy oraz rok wydania zostały znormalizowane do przedziału $[0, 1]$.

3.1 Dobór kroku uczenia

Hesjan dla metody najmniejszych kwadratów wynosi $A^T A$. Na podstawie obliczeń numerycznych wyznaczono ekstremalne wartości własne tej macierzy:

$$\lambda_{\min} = 4,5766, \quad \lambda_{\max} = 9,8235 \cdot 10^6$$

Współczynnik uwarunkowania wynosi $\kappa \approx 2,15 \cdot 10^6$. Stałą uczącą α (zapewniającą teoretycznie najszybszą zbieżność) wyznacza się ze wzoru:

$$\alpha = \frac{2}{\lambda_{\min} + \lambda_{\max}} \approx 2,0359 \cdot 10^{-7}$$

Niezwykle mała wartość kroku wynika bezpośrednio z bardzo złego uwarunkowania macierzy cech.

3.2 Porównanie wyników

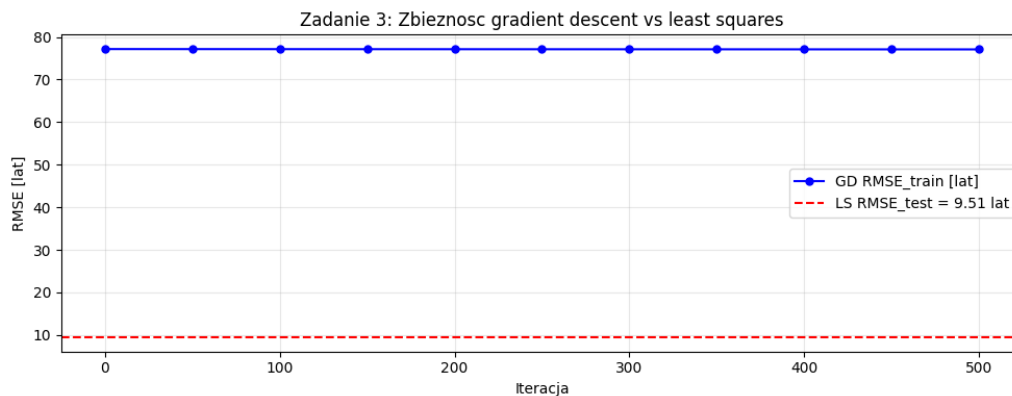
Wyniki działania obu metod zebrano w Tabeli 2.

Tabela 2: Porównanie metody najmniejszych kwadratów (dokładne rozwiązanie) oraz metody gradientu prostego (po 500 iteracjach).

Kryterium	Metoda LS	Metoda GD
RMSE na zbiorze testowym	9,5102 lat	77,1863 lat
Czas obliczeń	1,10 s	7,29 s
Złożoność teoretyczna	$\mathcal{O}(nd^2 + d^3)$	$\mathcal{O}(M \cdot nd)$ (dla M iteracji)

- **Dokładność predykcji:** Metoda najmniejszych kwadratów daje błąd testowy na poziomie 9,51 roku. Z kolei metoda gradientu prostego po 500 iteracjach osiąga zaledwie 77,19 roku (błąd trenujący zmniejszył się w tym czasie jedynie z 77,17 do 77,09 lat). Wynika to z faktu, że przy tak małym kroku α spadek błędu jest skrajnie wolny, do osiągnięcia zbliżonej dokładności potrzeba by milionów iteracji.
- **Czas obliczeń:** Metoda najmniejszych kwadratów wykonała się w 1,10 s, natomiast 500 iteracji GD trwało 7,29 s. GD jest wolniejszy, ponieważ w każdej iteracji musi przemnożyć duże macierze o rozmiarze $463\,715 \times 91$.
- **Złożoność obliczeniowa:** Teoretycznie dla bardzo dużych d gradient prosty mógłby być korzystniejszy, ponieważ jego złożoność rośnie liniowo z d , podczas gdy LS wymaga obliczenia $A^T A$ (koszt $\mathcal{O}(nd^2)$) oraz odwrócenia tej macierzy (koszt $\mathcal{O}(d^3)$). Jednak w tym przypadku $d = 91$ jest małe, więc narzut na operacje macierzowe w LS jest znikomy, a powolna zbieżność GD eliminuje go z praktycznego zastosowania.

Wykres zbieżności błędu RMSE dla metody gradientowej w porównaniu do linii odniesienia LS przedstawiono na Wykresie 4.



Wykres 4: Porównanie zbieżności błędu RMSE w kolejnych iteracjach metody gradientu prostego z ostatecznym wynikiem metody najmniejszych kwadratów.

4 Wnioski

1. Uwarunkowanie Heszjana ma kluczowe znaczenie dla szybkości zbieżności metody gradientu prostego. Zastosowanie dekompozycji Choleskiego jako techniki preconditioningu pozwoliło zmniejszyć liczbę iteracji z 115 do zaledwie 1, sprowadzając problem do idealnie uwarunkowanego.
2. W problemach o nieliniowej strukturze i wielu stopniach swobody, takich jak wiszący łańcuch, stały krok uczenia bardzo łatwo prowadzi do rozbieżności i eksplozji energii. Konieczne jest stosowanie dynamicznych metod doboru kroku, np. przeszukiwania liniowego Armijo, które gwarantują stabilność i zbieżność do fizycznego minimum.
3. W przypadku klasycznej regresji liniowej na dużych zbiorach danych, gdy liczba cech d jest stosunkowo mała, bezpośrednie rozwiązanie układu normalnego za pomocą metody najmniejszych kwadratów jest nieporównywalnie szybsze i dokładniejsze niż gradient prosty. Bardzo złe uwarunkowanie macierzy cech wymusza stosowanie ekstremalnie małego kroku uczenia, przez co gradient prosty staje się bezużyteczny.

Literatura

- [1] Wikipedia, *Gradient descent*, https://en.wikipedia.org/wiki/Gradient_descent (dostęp: 10 czerwca 2026 r.).